2520030117
K.Vaishali

PRACTICAL 9

Yep bro ﷺ **P-9**. I'll give it in the **same practical-record format**: Aim → Requirements → Theory → Procedure → Programs → Commands → Output → Result. You can copy this directly.

P-9: Linux File I/O and I/O Redirection

1. AIM

To implement a file copy utility using low-level Linux file I/O system calls such as `open()`, `read()`, `write()`, `lseek()` and `close()`, compare its performance with standard library functions such as `fopen()`, `fread()` and `fwrite()`, and implement standard input/output redirection using file descriptors and `dup2()`.

2. REQUIREMENTS

- Ubuntu/Linux operating system
- GCC compiler
- Terminal
- C programming language
- Linux system calls and standard I/O library functions

PART A — FILE COPY USING SYSTEM CALLS

3. THEORY

Linux provides low-level file I/O system calls to directly access files.

Function	Purpose
`open()`	Opens or creates a file

`read()` Reads data from a file

`write()` Writes data to a file

`lseek()` Changes the file offset

`close()` Closes the file

A file is accessed using a **file descriptor**, which is an integer returned by `open()`.

File descriptor values

0 → Standard Input
1 → Standard Output
2 → Standard Error

4. PROCEDURE

Step 1: Create the practical folder

mkdir p9
cd p9

Step 2: Create the source file

nano source.txt

Enter:

This is a file copy demonstration using Linux system calls.

Save:

Ctrl + O → Enter → Ctrl + X

5. LOW-LEVEL FILE COPY PROGRAM

Create:

nano lowlevel_copy.c

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
```

```
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2],
                       O_WRONLY | O_CREAT | O_TRUNC,
                       0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0)
        perror("Error reading file");

    close(source);
    close(destination);
```

```
    printf("File copied successfully using low-level I/O.\n");

    return 0;
}
```

6. EXECUTION

Compile:

gcc lowlevel_copy.c -o lowlevel_copy

Run:

./lowlevel_copy source.txt destination.txt

Output

File copied successfully using low-level I/O.

Check the copied file:

cat destination.txt

PART B — STANDARD LIBRARY FILE COPY

7. PROGRAM USING `fopen()`, `fread()`, `fwrite()`

Create:

nano standard_copy.c

Code

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *source, *destination;
    char buffer[1024];
    size_t bytesRead;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
```

```
        return 1;
    }

    source = fopen(argv[1], "rb");

    if (source == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");

    if (destination == NULL)
    {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }
    while ((bytesRead = fread(buffer, 1, sizeof(buffer), source)) > 0)
    {
        fwrite(buffer, 1, bytesRead, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard I/O.\n");

    return 0;
}
```

Compile:

```
gcc standard_copy.c -o standard_copy
```

Run:

```
./standard_copy source.txt destination2.txt
```

Output

```
File copied successfully using standard I/O.
```

8. PERFORMANCE COMPARISON

Use the Linux `time` command.

Low-level version

time ./lowlevel_copy source.txt copy1.txt

Standard library version

time ./standard_copy source.txt copy2.txt

The exact execution time depends on the system and file size.

Comparison

Low-Level I/O	Standard Library I/O
`open()`	`fopen()`
`read()`	`fread()`
`write()`	`fwrite()`
`close()`	`fclose()`
Uses file descriptors	Uses `FILE *`
Direct system calls	Library-level interface

PART C — I/O REDIRECTION USING `dup2()`

9. THEORY

The `dup2()` system call duplicates a file descriptor.

It can be used to redirect standard input and output.

Normally:

Keyboard
↓
stdin (0)
↓
Program
↓
stdout (1)
↓
Screen

After redirection:

input.txt
↓
stdin (0)
↓
Program
↓
stdout (1)
↓
output.txt

10. PROGRAM USING dup2( )

Create:

nano redirect.c

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int inputFile, outputFile;
    char buffer[100];
    ssize_t bytesRead;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile < 0)
    {
        perror("Error opening input file");
        return 1;
    }

    outputFile = open("output.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (outputFile < 0)
    {
        perror("Error opening output file");
```

```
        close(inputFile);
        return 1;
    }

    dup2(inputFile, STDIN_FILENO);
    dup2(outputFile, STDOUT_FILENO);
    close(inputFile);
    close(outputFile);

    while ((bytesRead = read(STDIN_FILENO,
                             buffer,
                             sizeof(buffer))) > 0)
    {
        write(STDOUT_FILENO, buffer, bytesRead);
    }

    return 0;
}
```

11. EXECUTION OF dup2( ) PROGRAM

Create input:

nano input.txt

Enter:

This is an example of input redirection.
dup2() redirects standard input and output.

Save and exit.

Compile:

gcc redirect.c -o redirect

Run:

./redirect

Nothing will appear on the terminal because **stdout has been redirected to `output.txt`**.

Check:

cat output.txt

Output

This is an example of input redirection.
dup2() redirects standard input and output.

12. DEMONSTRATE SHELL I/O REDIRECTION

The shell can perform redirection using:

cat < input.txt > shell_output.txt

Then:

cat shell_output.txt

Output:

This is an example of input redirection.
dup2() redirects standard input and output.

Explanation

< input.txt

redirects **standard input** from the file.

> shell_output.txt

redirects **standard output** to the file.

The shell opens the required files and sets up the appropriate file descriptors before executing the command.

13. RESULT

The file copy utility was successfully implemented using low-level Linux system calls `open()`, `read()`, `write()`, `lseek()` and `close()`. Its performance was compared with standard library functions `fopen()`, `fread()`, `fwrite()` and `fclose()`.

Standard input and output redirection was also successfully implemented using `dup2()`, demonstrating the basic mechanism used by Linux shells for I/O redirection.

```
vaishali@LAPTOP-QQD0F0TI:~$ mkdir -p Practical-10
vaishali@LAPTOP-QQD0F0TI:~$ cd /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/Practical-10
-bash: cd: /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/Practical-10: No such file or directory
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux" > original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -l original.txt
-rw-r--r-- 1 vaishali vaishali 12 Oct 30  2026 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
  File: original.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 12213       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:36:45.457805602 +0000
 Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$
```

```
vaishali@LAPTOP-QQD0F0TI:~$ ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
  File: original.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 12213       Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:37:05.452444785 +0000
 Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48    Inode: 12242       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
 Birth: 2026-10-30 14:37:57.009792289 +0000
vaishali@LAPTOP-QQD0F0TI:~$ find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
./symlink.txt
./hardlink.txt
./original.txt
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux World" > mmapfile.txt
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
vaishali@LAPTOP-QQD0F0TI:~$ nano mmap_program.c
vaishali@LAPTOP-QQD0F0TI:~$ gcc mmap_program.c -o mmap_program
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
vaishali@LAPTOP-QQD0F0TI:~$ ./mmap_program
-bash: ./mmap_program: No such file or directory
```

```
* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

  https://ubuntu.com/workshop

This message is shown once a day. To disable it please create the
```

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

  * Documentation:  https://help.ubuntu.com
  * Management:     https://landscape.canonical.com
  * Support:        https://ubuntu.com/pro

  System information as of Mon Sep 28 08:12:14 UTC 2026

   System load:  0.08                Processes:             58
   Usage of /:   0.3% of 1006.85GB   Users logged in:       0
   Memory usage: 6%                  IPv4 address for eth0: 172.17.254.203
   Swap usage:   0%

  * Canonical Workshop gives developers fast, composable, reproducible, and
    secure developer environments that are perfect for agentic workflows.

    https://ubuntu.com/workshop

This message is shown once a day. To disable it please create the
```